

i am attempting to make a procedural building generator in the unity editor, similar to how houdini can set something up like this. i want to be able to draw a line and generate a building, then select certain features of that building (like a group of windows) and replace it with a unique feature (so if i selected two horizontally aligned windows on the first floor and clicked replace it would put a door in or similar things). however, an additional thing id like to implement is the usage of a formal grammar to define how houses should generate. ideally, id like this to work in two parts: a procedural anything generator that lets you define grammars, and concrete implementations for buildings, trees, etc. now, if i am able to implement both of these in one (a single procedural generator that can handle buildings and trees and anything else as long as the grammar is defined) i would prefer doing that. however, i want to prioritize ease of use in the unity editor. additionally, i would like this to be very scalable. for example, i could define a generator that makes buildings, then set this up as an element in a generator that makes towns. in summary, i want to be able to define paths for the buildings, define feature generation using a formal grammar, customize building elements according to the formal grammar and selected user choices, and recursively stack generators on top of each other for maximum scalability. finally, id like to be able to bake these models into an actual fbx for use (hopefully a model that is as optimized as the base meshes provided for the grammar alphabet). i am not looking for code here, but instead recommendations on structure, theoretical implementation choices, and a work plan. i want this to be made as robustly and cleanly as possible, along with documentation, so i want to be sure to think everything through.

This is a very solid goal, and you're thinking about it at the *right* level already. What you're describing is essentially a **Houdini-style shape grammar system embedded in Unity**, with editor-first affordances and composability as a core principle.

I'll break this into **theoretical framing, system architecture, editor UX design, grammar choices, scalability/composability, baking & optimization**, and finally a **concrete work plan**. I'll stay deliberately code-free and focus on decisions and structure.

1. Conceptual Framing (what you're really building)

You are not "making a building generator."

You are building:

A domain-agnostic procedural composition system driven by formal grammars, with spatial semantics and editor interaction.

Buildings, trees, towns, etc. are **grammar instances**, not special cases.

This framing matters because it pushes you toward:

- **Separation of grammar evaluation from geometry**
- **Editor tooling as a first-class feature**
- **Composable generators**
- **Late binding of meshes/materials**
- **Deterministic but overridable generation**

Think in terms of:

- *Symbols*
- *Rules*
- *Derivations*
- *Scopes*
- *Constraints*
- *Spatial context*

2. Core Architectural Layers (clean separation)

A robust system here wants **at least five layers**, each with a single responsibility.

Layer 1 — Grammar Core (engine-agnostic)

This layer knows *nothing* about Unity meshes.

Responsibilities:

- Symbol definitions
- Grammar rules
- Rule selection (deterministic, weighted, conditional)
- Derivation trees
- Parameters & attributes
- Scopes (important for editor selection later)

Key idea:

- Output is a **typed derivation graph**, not geometry.

This lets:

- Buildings, trees, roads all use the same machinery
- You unit test grammar logic outside of rendering

Layer 2 — Spatial Interpretation Layer

This layer turns **grammar symbols into spatial operations**, but still not meshes.

Responsibilities:

- Interpret symbols as:
 - Extrude
 - Subdivide
 - Repeat
 - Offset
 - Replace
- Maintain coordinate frames (local axes, floors, façades)
- Maintain *semantic groups* (e.g. "these windows are aligned")

Output:

- A **procedural layout graph** (volumes, faces, segments, feature slots)

This is where:

- "Two horizontally aligned windows" becomes a *queryable structure*
- Selection logic lives (floors, rows, façades, groups)

Layer 3 — Geometry Binding Layer

This layer binds abstract features to **actual assets**.

Responsibilities:

- Map grammar terminals → mesh providers
- Handle:
 - Variants
 - LODs
 - Materials
 - Instancing rules
- Define replacement compatibility (window → door constraints)

Important:

- Grammar doesn't know what an FBX is
- Geometry binding can be swapped or reauthored without changing rules

Layer 4 — Editor Interaction Layer (Unity-only)

This is where you differentiate from Houdini *for Unity users*.

Responsibilities:

- Path drawing
- Face/feature selection
- Rule overrides
- Visual debugging of grammar derivations
- Live regeneration
- Override persistence

This layer should operate by:

- Editing **parameters and rule overrides**, not geometry directly

Layer 5 — Baking & Optimization Layer

Responsible for turning procedural output into production-ready assets.

Responsibilities:

- Collapse instances where appropriate
- Preserve instancing where possible
- Merge meshes by material
- Generate secondary UVs
- Export FBX / prefab variants

3. Grammar Choice: What kind of “formal grammar” you actually want

Do NOT use a pure CFG

Context-free grammars alone are *not enough* for buildings.

You want a **Parameterized, Context-Sensitive Shape Grammar**, similar to:

- CGA Shape Grammar (CityEngine)
- Split Grammars
- Attribute Grammars

Key features you need:

- Parameters on symbols (width, floor index, orientation)
- Access to parent scope data
- Spatial predicates (“if on ground floor”)
- Rule priorities and weights

Conceptually:

scss

```
Facade(width, floor)
→ Split(X)
```

[Window | Window | Door | Window]

Your grammar rules should:

- Be data-driven (ScriptableObjects or JSON)
- Support:
 - Conditional rules
 - Weighted alternatives
 - User-forced replacements

 Copy code

4. Editor UX: how this actually feels to use

Ease of use lives or dies here.

4.1 Path-based generation

- User draws:
 - Polyline
 - Closed shape
- Path becomes:
 - Primary grammar seed
 - Carries metadata (street-facing, interior-facing, corner)

This is a **root symbol initializer**, not geometry.

4.2 Feature Selection & Replacement

Your idea of selecting windows and replacing them is key—and nontrivial.

This requires:

- Every generated feature to retain:
 - Symbol ID
 - Scope ID
 - Rule origin
- A way to *query groups*:
 - Same rule
 - Same parent

- Same floor
- Same alignment

Replacement workflow:

1. User selects features
2. Editor resolves selection → grammar scope
3. User chooses a replacement rule
4. System:
 - Overrides grammar at that scope
 - Regenerates *only affected subtree*

This is why derivation trees matter.

4.3 Debug Visualization

You will absolutely want:

- Grammar tree viewer
- Color-coded scopes
- Rule application highlights
- Regeneration diffs

Without this, the system becomes unmaintainable.

5. Scalability & Recursive Composition (towns, districts, worlds)

Your instinct to allow generators inside generators is **correct**.

Conceptually:

- A generator produces **symbols**, not meshes
- Some symbols represent *child generators*

Example:

mathematica

Town

→ RoadNetwork

- **Plot**
- **BuildingGenerator**

Each generator:

- Has inputs
- Emits symbols or layouts
- Can be evaluated independently

 Copy code

This gives you:

- Towns made of buildings
- Buildings made of façades
- Façades made of features

Important rule:

Never let a generator directly depend on the geometry output of another generator.

They should only exchange **semantic/layout data**.

6. Baking & Optimization Strategy (critical for real use)

You're right to care about FBX quality early.

Key principles:

- Preserve instancing where possible
- Avoid vertex baking too early
- Allow multiple bake profiles

Baking modes:

- **Editor procedural**
 - Fully parametric
 - Editable
- **Runtime procedural**
 - Reduced flexibility
 - Cached layouts
- **Baked static**

- Mesh merged
- LODs generated
- FBX export

You should aim for:

- Geometry instancing during generation
- Optional mesh merge per material during bake
- Export as:
 - Single FBX
 - Or prefab with submeshes

7. Documentation Philosophy (important if this grows)

Document **concepts**, not APIs first.

You'll want:

- Grammar mental model
- Symbol lifecycle
- Scope & override rules
- Editor workflows
- Performance characteristics

Think:

"How would a level designer understand *why* this building looks the way it does?"

8. Recommended Work Plan (pragmatic and safe)

Phase 1 — Grammar Core Prototype

- Minimal grammar engine
- Text-based rules
- Outputs derivation trees only
- No Unity dependency

Phase 2 — Spatial Layout Interpreter

- Boxes, faces, segments
- Split / repeat / extrude
- Debug drawing in Unity

Phase 3 — Editor MVP

- Path drawing
- Regenerate button
- Simple overrides
- Grammar tree debug UI

Phase 4 — Feature Selection & Replacement

- Selection grouping logic
- Scope-aware overrides
- Partial regeneration

This is the hardest and most important phase.

Phase 5 — Composability

- Generators referencing generators
- Town → building pipeline

Phase 6 — Baking & Export

- Mesh consolidation
- FBX export
- LOD & material workflows

9. Final Strategic Advice

- **Resist premature mesh optimization** until grammar & scopes are rock solid

- **Design for introspection**—you must be able to ask “why did this generate?”
- **Never hardcode buildings**—build tools that *build buildings*
- Treat Unity as:
 - A *host*
 - An *editor*
 - Not the logic core

If you want, next we can:

- Compare CGA vs L-systems vs your custom grammar
- Sketch a concrete data model (symbols, rules, scopes)
- Design the exact editor UX flow
- Or stress-test this design against edge cases (curved paths, corners, mixed styles)

This is a very real, very ambitious system—and you’re approaching it like someone who could actually finish it.



yes lets do all of these steps be as detailed as possible

Great — then let’s treat this like a **full systems design document** for a Houdini-class procedural framework inside Unity. I’ll be extremely explicit, including tradeoffs, failure modes, and *why* certain choices matter. This will be long, but intentionally so; this is the kind of thinking that saves months later.

I’ll proceed in this order (mirroring what you asked):

1. Formal grammar comparison & selection
2. Concrete data model (symbols, rules, scopes, derivations)
3. Spatial semantics & layout system
4. Editor UX & interaction flows
5. Recursive generators & composability
6. Baking, instancing, and FBX export
7. Failure cases & scalability constraints
8. Phased implementation roadmap (very concrete)

Throughout, I'll anchor things to **Unity editor reality**, not abstract theory.

1. Grammar Models: What Actually Works for Architecture

Let's clear something up early:

You do **not** want a traditional L-system or pure CFG.

Those are attractive on paper and terrible in practice for buildings.

Below is a comparison that leads to the right hybrid.

1.1 Context-Free Grammars (CFGs)

Pros

- Simple
- Deterministic derivation trees
- Easy to reason about

Cons

- No spatial awareness
- No access to parent context
- Impossible to say "ground floor does X"

Verdict

✗ Not viable alone

1.2 L-Systems

Pros

- Great for plants
- Natural recursion
- Symbol rewriting is elegant

Cons

- No inherent spatial constraints
- Weak for discrete architectural features
- Hard to interactively override specific instances

Verdict

✗ Trees only, not buildings

1.3 Shape Grammars (CGA / Split Grammars)

Pros

- Designed for architecture
- Spatial operators built-in
- Natural mapping to extrusions & subdivisions
- Proven (CityEngine)

Cons

- Traditionally not interactive
- Needs adaptation for editor selection
- Needs attribute system

Verdict

✓ Correct foundation

1.4 Attribute Grammars (Critical)

This is the missing piece.

You need:

- Symbols with parameters
- Rules that inspect parameters
- Symbols that emit attributes upward/downward

Examples:

- `floorIndex`
- `isCorner`
- `widthRemaining`

- streetFacing

Verdict

 Mandatory

1.5 Final Grammar Choice (What You Want)

Parameterized, Attribute-Driven Shape Grammar

Core properties:

- Symbols carry parameters
- Rules are conditional
- Rules emit spatial operators
- Grammar evaluation produces a **derivation tree**
- Tree nodes retain identity

This is what enables:

- Feature replacement
- Partial regeneration
- Editor selection
- Recursive composition

2. Core Data Model (This Is the Spine of Everything)

This section is *the most important*. If you get this wrong, nothing else will scale.

2.1 Symbol Definition

A **symbol** is *not* geometry.

It is:

- A semantic instruction
- With parameters
- And a lifecycle

Symbol fields

- `SymbolType` (Facade, Window, Door, Roof, etc.)
- `Parameters` (dictionary or typed struct)
- `ScopeID`
- `ParentSymbolID`
- `ChildSymbols[]`
- `RuleAppliedID`

Symbols must be:

- Serializable
- Stable across regenerations (ID persistence matters)

2.2 Grammar Rules

A rule answers:

"If I see symbol *X* *with context Y*, how do I expand it?"

Rule structure

- Input symbol type
- Conditions (predicates over parameters + scope)
- Probability / priority
- Expansion instructions

Expansion instructions produce:

- New symbols
- Spatial operators
- Parameter mutations

Important:

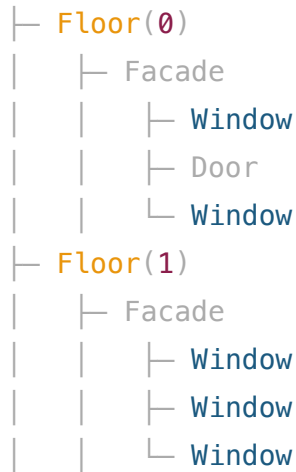
- Rules should never directly create meshes

2.3 Derivation Tree (Non-negotiable)

Every generation produces a **derivation tree**:

sql

Building



Why this matters:

- Selection maps to nodes
- Overrides attach to nodes
- Partial regeneration becomes possible

Never flatten this prematurely.

2.4 Scope System (The Secret Weapon)

 Copy code

Scopes are **semantic groupings**.

Examples:

- Entire building
- Single floor
- Single facade
- Window row
- Custom user-defined scope

Each symbol belongs to exactly one scope, but scopes may nest.

Scopes enable:

- "Select all windows on floor 1"
- "Replace this group only"
- Grammar overrides without ambiguity

3. Spatial Interpretation Layer (From Symbols to Layouts)

This is where grammar becomes geometry *without meshes yet*.

3.1 Abstract Geometry Primitives

You want a **layout graph**, not triangles.

Primitives:

- Oriented bounding boxes
- Faces
- Segments
- Polylines
- Volumes

Each carries:

- Local coordinate frame
- Symbol reference
- Scope reference

3.2 Spatial Operators (Grammar Emits These)

Your grammar expansions should use operators like:

- `Extrude(height)`
- `Split(axis, sizes)`
- `Repeat(axis, minSize, maxCount)`
- `Inset(amount)`
- `Align(edge)`
- `Offset(distance)`

These operators:

- Modify layout primitives
- Produce child layout nodes

- Preserve symbol identity

3.3 Why This Layer Must Exist

If you skip this and go straight to meshes:

- Selection becomes impossible
- Replacement becomes destructive
- Baking becomes irreversible

Think:

Houdini SOPs, not Unity MeshFilters

4. Editor UX & Interaction (Where This Lives or Dies)

Your system is only as good as its **editor ergonomics**.

4.1 Path-Based Generation

User draws:

- Open polyline → strip building
- Closed polyline → block building

Path becomes:

- Root symbol initializer
- Provides:
 - Orientation
 - Length
 - Corner flags
 - Street-facing info

This is a *semantic seed*, not geometry.

4.2 Selection Semantics

When user clicks geometry:

1. Raycast hits mesh
2. Mesh maps to layout primitive
3. Layout primitive maps to symbol
4. Symbol maps to scope

Selections can expand to:

- Same symbol type
- Same scope
- Same rule origin
- Same spatial alignment

This is why IDs and scopes matter.

4.3 Replacement Workflow (Critical)

User intent

"Replace these two windows with a door"

System behavior

1. Identify smallest common scope
2. Validate replacement compatibility
3. Attach override rule at that scope
4. Regenerate subtree
5. Preserve everything else

Overrides should be:

- Explicit
- Visible
- Removable

4.4 Grammar Debugging Tools

You *will* need:

- Tree view of derivation

- Highlight rule origins
- Visualize scopes
- Step-through generation

Without this, debugging is impossible.

5. Recursive Generators & Composability

This is where your system becomes *world-scale*.

5.1 Generators as Symbols

A generator is just:

- A symbol that expands into another grammar evaluation

Example:

mathematica

Town

→ **RoadNetwork**

→ **Plot**

→ **BuildingGenerator**

Each sub-generator:

- Has inputs
- Emits symbols/layouts
- Does not expose geometry internals

 Copy code

5.2 Rules for Safe Composition

Hard constraints:

- Parent generators cannot inspect child geometry
- Communication is via attributes only
- Child generators must be deterministic

This keeps the system debuggable.

5.3 Town Example

- Town grammar defines:
 - Roads
 - Blocks
 - Plots
- Plot symbol invokes building grammar
- Building grammar uses plot attributes

This lets:

- Buildings adapt to context
- Towns remain editable

6. Baking, Optimization, and FBX Export

This must be designed early.

6.1 Instancing Strategy

During generation:

- Everything instanced
- Mesh identity preserved

During bake:

- Merge only when beneficial
- Preserve instancing where possible

6.2 Bake Profiles

Have multiple profiles:

- **Preview**
- **Runtime**
- **Export**

Each controls:

- Mesh merging
- LOD generation
- Collider generation
- Lightmap UVs

6.3 FBX Export Philosophy

Export:

- One FBX per generator output
- Or prefab with baked meshes

Guarantee:

- No procedural dependencies
- Matches base mesh quality

7. Failure Modes & Constraints (Be Honest)

You will hit these:

- Grammar explosion (too many symbols)
- Rule ambiguity
- Performance during regeneration
- Editor undo/redo complexity

Mitigations:

- Rule priorities
- Cached subtrees
- Explicit regeneration boundaries
- Deterministic seeds

8. Phased Implementation Plan (Concrete)

Phase 1 — Grammar Core (No Unity)

- Symbols
- Rules

- Derivation trees
- Unit tests

Phase 2 — Spatial Interpreter

- Boxes, splits, repeats
- Debug drawing
- No meshes

Phase 3 — Unity Editor MVP

- Path drawing
- Live regeneration
- Grammar visualization

Phase 4 — Selection & Overrides

- Scope resolution
- Partial regeneration
- Replacement UI

Phase 5 — Composability

- Generator nesting
- Town → building

Phase 6 — Baking & Export

- Mesh binding
- FBX export
- Optimization passes

Final Thought

What you're proposing is *not* overengineering — it's **correct engineering** for this problem.

You are essentially building:

A Houdini-like procedural modeling kernel with Unity-native ergonomics.

If you want next, I can:

- Design an example **building grammar** end-to-end
- Sketch **data schemas** (ScriptableObjects vs JSON)
- Design the **selection & override algorithm in detail**
- Or stress-test this against extreme cases (curved buildings, towers, ruins)

Just tell me which layer you want to drill into next.



what is an sample example of a grammar to make a building

Great question — this is the right moment to make everything concrete.

Below is a **complete, realistic sample building grammar**, written in a **pseudo-CGA / attribute-shape-grammar style** that matches the system we've been designing. This is **not code**, but it *is executable in spirit*. I'll annotate heavily so you can see *why* each piece exists and how it supports editor overrides, selection, and scalability.

1. High-Level Grammar Overview

We'll build a **street-facing rectangular building** with:

- Variable number of floors
- Ground floor with doors
- Upper floors with windows
- Roof type variation
- Replaceable façade features

Hierarchy:

scss

Building

├ Mass

| └ Floor(0)

| | └ Facade (ground)

```

|   |─ Floor(1..n)
|   |   └─ Facade (upper)
└─ Roof

```

2. Symbol Definitions (Semantic Alphabet)

These are **grammar symbols**, not meshes.

scss

```

Building(path, depth)
Mass(width, depth, height)
Floor(index, height)
Facade(width, height, floorIndex)
Bay(width, floorIndex)
Window(width, height)
Door(width, height)
Roof(width, depth)

```

Each symbol:

- Carries parameters
- Belongs to a scope
- Is selectable

3. Root Rule (Path → Building)

 Copy code

This initializes generation from editor input.

scss

```

Rule Building(path, depth):
  width = path.length
  floors = randomInt(2, 6)

  → Mass(width, depth, floors * FLOOR_HEIGHT)

```

Editor override examples:

- User sets number of floors
- User locks width

 Copy code

4. Mass Decomposition

Split vertically into floors.

arduino

```
Rule Mass(width, depth, height):
    floors = height / FLOOR_HEIGHT

    → Split(Y):
        Floor(0, FLOOR_HEIGHT)
        Floor(1..floors-1, FLOOR_HEIGHT)
```

Important:

- Each Floor becomes a **scope**
- Floor index propagates downward

5. Floor → Facade

 Copy code

Each floor creates a façade facing the street.

pgsql

```
Rule Floor(index, height):
    → Facade(width, height, index)
```

At this point:

- The facade is a rectangular surface
- No windows yet

 Copy code

6. Facade Rules (Ground vs Upper Floors)

This is where attribute grammars matter.

Ground Floor

arduino

```
Rule Facade(width, height, floorIndex)
  if floorIndex == 0:

    → Split(X):
      Bay(BAY_WIDTH, floorIndex)
      Bay(BAY_WIDTH, floorIndex)
      Bay(BAY_WIDTH, floorIndex)
```

Upper Floors

arduino

```
Rule Facade(width, height, floorIndex)
  if floorIndex > 0:

    → Repeat(X, minWidth = BAY_WIDTH):
      Bay(BAY_WIDTH, floorIndex)
```

Why this matters:

- Different logic for ground floor
- Bays become **groupable window units**

 Copy code

7. Bay → Feature Selection

Each bay decides what it contains.

Ground Floor Bay

arduino

```
Rule Bay(width, floorIndex)
  if floorIndex == 0:

    choose weighted:
```

```
0.6 → Window(width * 0.8, height * 0.6)
```

```
0.4 → Door(width * 0.9, height * 0.9)
```

Upper Floor Bay

SCSS

```
Rule Bay(width, floorIndex)
```

```
  if floorIndex > 0:
```

```
    → Window(width * 0.7, height * 0.5)
```

This is the **key replacement point**:

- Selecting two windows → replacing with a door
- User override swaps this rule at the Bay scope

 Copy code

8. Windows and Doors (Terminals)

These are **terminals**, meaning they bind to geometry later.

CSS

```
Rule Window(width, height):
```

```
  → Place(WindowMesh, width, height)
```

```
Rule Door(width, height):
```

```
  → Place(DoorMesh, width, height)
```

Editor notes:

- WindowMesh is a *binding*, not hardcoded
- Variants can be swapped

 Copy code

9. Roof Generation

Roof is independent and replaceable.

CSS

```
Rule Mass(width, depth, height):
  → Roof(width, depth)
```

SCSS

```
Rule Roof(width, depth):
  choose weighted:
    0.7 → FlatRoof(width, depth)
    0.3 → GabledRoof(width, depth)
```

10. Why This Grammar Works Well

✓ Interactive Replacement

- Windows belong to Bay scopes
- Replacing a rule at a scope regenerates locally

✓ Scalability

- Floors are independent
- Facades can be reused
- Bay logic reusable for shops, apartments, etc.

✓ Composability

- Building is just a symbol
- Can be spawned by a Town grammar

 Copy code

11. Example Editor Interaction

User action

Select two adjacent ground-floor windows → Replace → Door

System behavior

1. Finds common parent: Facade(floorIndex=0)
2. Identifies selected Bay symbols
3. Overrides Bay → Door rule for those scopes

4. Regenerates only those bays

Everything else stays unchanged.

12. Visual Grammar Tree (Simplified)

sql

Building

└─ Mass

└─ Floor(0)

└─ Facade

└─ Bay → Door

└─ Bay → Window

└─ Bay → Door ← user override

└─ Floor(1)

└─ Facade

└─ Bay → Window

└─ Bay → Window

└─ Bay → Window

└─ Roof

13. Extending This Grammar

You can add:

- Corner bays
- Balconies
- Fire escapes
- Shops
- Signage
- Damage/decay

Without changing the core system.

 Copy code

Key Takeaway

This grammar:

- Is **formal**
- Is **editable**
- Is **interactive**
- Scales from buildings → cities
- Matches Houdini-style thinking
- Fits Unity editor workflows

If you want next, I can:

- Rewrite this grammar in a **more formal BNF-like syntax**
- Show how **tree grammars** would differ
- Design a **tree grammar example**
- Or map this grammar directly to your **Unity editor data model**

Just tell me where you want to zoom in.



write this more formally, i want to see what input i should expect to give to the generator

Below is the **same building grammar** rewritten in a formal, generator-input-oriented specification.

Think of this as **what the user (or you) would actually author** to drive the procedural system.

I'll present it in four layers, because a *real* system needs all four:

1. **Grammar declaration (metadata & parameters)**
2. **Symbol definitions (alphabet)**
3. **Rule definitions (formal, conditional, weighted)**
4. **Geometry binding (separate, late-bound)**

I will use a **clean, declarative pseudo-DSL** inspired by CGA + attribute grammars + what works well in Unity tooling.

1. Grammar Declaration

This is the *entry point* and global configuration.

markdown

```
grammar BuildingGrammar {

    version = 1.0
    unit = meters

    parameters {
        FloorHeight = 3.0
        BayWidth    = 2.5
        MinFloors   = 2
        MaxFloors   = 6
    }

    entry = Building
}
```

What this means:

- The generator expects a **Building** symbol as root
- Parameters are exposed to the Unity editor
- Everything downstream can reference these

2. Symbol Alphabet (Formal Definitions)

Symbols are **typed, parameterized nonterminals or terminals**.

less

```
symbols {

    // Structural
    Building(path: Path, depth: Float)
    Mass(width: Float, depth: Float, height: Float)

    // Vertical decomposition
    Floor(index: Int, height: Float)
```

```
// Facade logic
Facade(width: Float, height: Float, floorIndex: Int)
Bay(width: Float, floorIndex: Int)

// Terminals
Window(width: Float, height: Float)
Door(width: Float, height: Float)
Roof(width: Float, depth: Float)
}
```

Key points:

- Every symbol has **explicit parameters**
- No symbol implies geometry
- This is the **alphabet** your generator understands

3. Rule Definitions (Formal Grammar Core)

Rules define **symbol rewriting**.

General form:

```
php-template

rule <RuleName>
  when <Predicate>
  produces <Expansion>
```

3.1 Root Rule: Building → Mass

```
arduino

rule BuildingToMass
  when Building(path, depth)
  produces {

    let width = length(path)
```

 Copy code

```

    let floors = randomInt(MinFloors, MaxFloors)

    Mass(
      width  = width,
      depth  = depth,
      height = floors * FloorHeight
    )
  }

```

Generator input required

- A Path (drawn in editor)
- A depth value

3.2 Mass → Floors

arduino

```

rule MassToFloors
  when Mass(width, depth, height)
  produces {

    let floorCount = height / FloorHeight

    split axis Y into floorCount {
      Floor(index = $index, height = FloorHeight)
    }
  }
}

```

Notes:

- \$index is an implicit split index
- Each Floor becomes its own **scope**

3.3 Floor → Facade

arduino

```
rule FloorToFacade
  when Floor(index, height)
  produces {

    Facade(
      width      = parent.width,
      height     = height,
      floorIndex = index
    )
  }
```

Context access:

- parent.width comes from Mass

3.4 Facade → Bays (Ground Floor)

arduino

```
rule GroundFloorFacade
  when Facade(width, height, floorIndex)
  if floorIndex == 0
  produces {

    split axis X by fixed BayWidth {
      Bay(width = BayWidth, floorIndex = floorIndex)
    }
  }
```

3.5 Facade → Bays (Upper Floors)

scss

```
rule UpperFloorFacade
  when Facade(width, height, floorIndex)
  if floorIndex > 0
  produces {
```

```

repeat axis X minWidth BayWidth {
  Bay(width = BayWidth, floorIndex = floorIndex)
}
}

```

Why two rules?

- This allows **clean overrides**
- Editor can disable one rule explicitly

3.6 Bay → Feature Selection (Ground Floor)

arduino

```

rule GroundFloorBay
  when Bay(width, floorIndex)
  if floorIndex == 0
  choose {

    0.6:
      Window(
        width  = width * 0.8,
        height = FloorHeight * 0.6
      )

    0.4:
      Door(
        width  = width * 0.9,
        height = FloorHeight * 0.9
      )
  }

```

This is the **primary user override point**.

3.7 Bay → Feature (Upper Floors)

arduino

```

rule UpperFloorBay
  when Bay(width, floorIndex)
  if floorIndex > 0
  produces {

    Window(
      width  = width * 0.7,
      height = FloorHeight * 0.5
    )
  }

```

3.8 Roof Generation

arduino

```

rule MassToRoof
  when Mass(width, depth, height)
  produces {

    Roof(width = width, depth = depth)
  }

```

arduino

```

rule RoofVariants
  when Roof(width, depth)
  choose {

    0.7:
      FlatRoof(width, depth)

    0.3:
      GabledRoof(width, depth)
  }

```

4. Geometry Binding (Separate File / Asset)

This is **not** part of the grammar.

scss

```
bindings {  
  
    Window      → Mesh("Window_A")  
    Door         → Mesh("Door_A")  
    FlatRoof     → Mesh("Roof_Flat")  
    GabledRoof   → Mesh("Roof_Gabled")  
}
```

Why separate?

- You can swap art without touching grammar
- Supports LODs, variants, themes

5. What the Generator Expects as Input

 Copy code

At runtime or in-editor, the generator needs:

Required

- Root symbol: Building
- Path (drawn spline / polyline)
- Depth

Optional Overrides

- Number of floors
- Rule disabling / forcing
- Replacement rules at scopes
- Asset bindings

6. Example Minimal Input (Editor Perspective)

yaml

Generate:

Symbol: Building

Path: <user-drawn polyline>

Depth: 8.0

Everything else flows from grammar.

7. Why This Is a *Good* Formal Input Format

 Copy code

- ✓ Explicit
- ✓ Deterministic
- ✓ Serializable
- ✓ Editor-friendly
- ✓ Supports partial regeneration
- ✓ Grammar-agnostic (buildings, trees, etc.)

Final Mental Model

The user does **not** author geometry.

The user authors **rules, symbols, and overrides**.

Geometry is a *consequence*.

If you want next, I can:

- Tighten this into an **EBNF-style formal grammar**
- Show how **user overrides are stored formally**
- Design the **scope resolution algorithm**
- Show a **tree grammar version** for vegetation

Just say the word.



tighten this into a formal grammar, and explain how i could serialize this input in the unity editor most easily and visually

Excellent — now we're at the point where this becomes an **actual authorable system**, not just a concept.

I'll do this in **two tightly coupled parts**:

PART I — A *Formal* Grammar Specification

I will:

- Make the grammar **precise enough to be machine-parseable**
- Keep it **authorable by designers**
- Preserve **Unity editor friendliness**

This is *not* CFG/BNF in the classical sense — it is a **parameterized attribute shape grammar**, but we *will* formalize it rigorously.

1. Grammar Meta-Syntax (EBNF-like)

This defines *what a valid grammar file looks like*.

go

```
Grammar      ::= "grammar" Identifier "{"
                GrammarHeader
                SymbolBlock
                RuleBlock
                BindingBlock?
                "}"
```

```
GrammarHeader ::= "entry" "=" Identifier
                ParameterBlock?
```

```
ParameterBlock ::= "parameters" "{" ParameterDecl* "}"
```

```
ParameterDecl ::= Identifier "=" Literal
```

```
SymbolBlock   ::= "symbols" "{" SymbolDecl* "}"
```

```
SymbolDecl    ::= Identifier "(" ParamList? ")"
```

```

ParamList      ::= ParamDecl ("," ParamDecl)*
ParamDecl      ::= Identifier ":" Type

RuleBlock      ::= "rules" "{" RuleDecl+ "}"

RuleDecl       ::= "rule" Identifier
                  "when" SymbolPattern
                  Condition?
                  "=>" Expansion

Condition       ::= "if" Expression

Expansion       ::= Production
                  | Split
                  | Repeat
                  | Choice

BindingBlock    ::= "bindings" "{" BindingDecl* "}"

```

This is your **formal grammar-of-grammars**.

2. Core Grammar Constructs (Formally Defined)

2.1 Symbol Pattern

go

```

SymbolPattern ::= Identifier "(" ArgList? ")"
ArgList       ::= Arg ("," Arg)*
Arg           ::= Identifier | "_"

```

Example:

arduino

Facade(width, height, floorIndex)

2.2 Expressions (Minimal, Deterministic)

mathematica

```

Expression ::= Identifier
            | Literal
            | Expression BinaryOp Expression
            | FunctionCall

BinaryOp    ::= "==" | ">" | "<" | ">=" | "<="

FunctionCall ::= Identifier "(" ArgList? ")"

```

No arbitrary code execution — important for safety & determinism.

 Copy code

2.3 Production (Direct Rewrite)

go

```

Production ::= Identifier "(" AssignmentList? ")"

AssignmentList ::= Assignment ("," Assignment)*
Assignment      ::= Identifier "=" Expression

```

2.4 Split Operator

mathematica

```

Split ::= "split" Axis
        "{" Production+ "}"

Axis   ::= "X" | "Y" | "Z"

```

Optional extended form:

bash

```
split X by FixedSize
split Y into Count
```

 Copy code

2.5 Repeat Operator

mathematica

```
Repeat ::= "repeat" Axis
        "min" Expression
        "{" Production "}"
```

2.6 Choice Operator (Weighted)

 Copy code

go

```
Choice ::= "choose" "{"
         Weight ":" Expansion
         + "}"
```

```
Weight ::= Float
```

3. Fully Formalized Building Grammar

Now rewritten strictly using the above rules.

 Copy code

3.1 Grammar Header

nginx

```
grammar Building {

    entry = Building
```

```
parameters {
  FloorHeight = 3.0
  BayWidth    = 2.5
  MinFloors   = 2
  MaxFloors   = 6
}
```

3.2 Symbols

pgsql

```
symbols {

  Building(path: Path, depth: Float)
  Mass(width: Float, depth: Float, height: Float)

  Floor(index: Int, height: Float)

  Facade(width: Float, height: Float, floorIndex: Int)
  Bay(width: Float, floorIndex: Int)

  Window(width: Float, height: Float)
  Door(width: Float, height: Float)

  Roof(width: Float, depth: Float)
  FlatRoof(width: Float, depth: Float)
  GabledRoof(width: Float, depth: Float)
}
```

3.3 Rules

less

```
rules {

  rule BuildingToMass
    when Building(path, depth)
```

```
=> Mass(  
  width = length(path),  
  depth = depth,  
  height = randomInt(MinFloors, MaxFloors) * FloorHeight  
)
```

rule MassToFloors

```
when Mass(width, depth, height)  
=> split Y {  
  Floor(index = $index, height = FloorHeight)  
}
```

rule FloorToFacade

```
when Floor(index, height)  
=> Facade(  
  width      = parent.width,  
  height     = height,  
  floorIndex = index  
)
```

rule GroundFacade

```
when Facade(width, height, floorIndex)  
if floorIndex == 0  
=> split X {  
  Bay(width = BayWidth, floorIndex = floorIndex)  
}
```

rule UpperFacade

```
when Facade(width, height, floorIndex)  
if floorIndex > 0  
=> repeat X min BayWidth {  
  Bay(width = BayWidth, floorIndex = floorIndex)  
}
```

rule GroundBay

```
when Bay(width, floorIndex)  
if floorIndex == 0  
=> choose {  
  0.6: Window(width = width * 0.8, height = FloorHeight * 0.6)  
  0.4: Door (width = width * 0.9, height = FloorHeight * 0.9)
```

```

    }

    rule UpperBay
    when Bay(width, floorIndex)
    if floorIndex > 0
    => Window(width = width * 0.7, height = FloorHeight * 0.5)

    rule MassToRoof
    when Mass(width, depth, height)
    => Roof(width = width, depth = depth)

    rule RoofVariant
    when Roof(width, depth)
    => choose {
        0.7: FlatRoof(width, depth)
        0.3: GabledRoof(width, depth)
    }
}

```

3.4 Bindings

```

rust

bindings {
    Window      -> Mesh("Window_A")
    Door        -> Mesh("Door_A")
    FlatRoof    -> Mesh("Roof_Flat")
    GabledRoof  -> Mesh("Roof_Gabled")
}

```

3.5 End Grammar

```

}

```

This grammar is now:

 Copy code

- Parseable
- Deterministic
- Serializable
- Editor-safe

PART II — How to Serialize This in Unity *Visually & Cleanly*

This is the *real* question.

1. Do NOT Store This as Raw Text Only

You *can*, but you shouldn't rely on it for editor UX.

Recommended Hybrid:

- Structured ScriptableObjects
- Optional DSL text view (advanced users)

2. Unity Serialization Strategy (Best Practice)

2.1 One Grammar = One ScriptableObject

swift

BuildingGrammar.asset

Contains:

- Entry symbol
- Parameters
- Symbol definitions
- Rules
- Bindings

 Copy code

 Copy code

3. Core ScriptableObject Types

GrammarAsset

- EntrySymbol
- ParameterList
- SymbolList
- RuleList
- BindingList

SymbolDefinition

- Name
- Parameter schema
- Category (Structural / Feature / Terminal)

RuleDefinition

- Input symbol
- Conditions (serialized expression tree)
- Expansion type:
 - Production
 - Split
 - Repeat
 - Choice
- Weight (optional)

Expression Serialization

This is important.

Instead of text:

- Use a **node-based expression tree**
- Operators: $=$, $>$, $<$, arithmetic
- Values: constants, parameters, symbol fields

This avoids:

- Code injection
- Runtime parsing bugs

4. Visual Rule Authoring (Unity Editor)

Recommended UX:

- **Node graph** for rules
- Dropdowns for symbols
- Visual split/repeat blocks
- Weighted choice sliders

Think:

Shader Graph, but for grammars

5. Scope & Override Serialization

Overrides should be stored as:

markdown

```
Override {  
    ScopeID  
    RuleID  
    ReplacementRuleID  
}
```

This allows:

- Undo/redo
- Partial regeneration
- Stable references

 Copy code

6. Geometry Binding Serialization

Bindings are easy:

- Symbol → Prefab or Mesh
- Variant list
- LOD options

Store separately so art teams can edit safely.

7. Why This Works Well in Unity

- ✓ Fully serializable
- ✓ Supports Undo
- ✓ Diff-friendly (YAML)
- ✓ Editor tooling friendly
- ✓ No runtime parsing required
- ✓ Designers don't touch code

Final Advice (Important)

If you do **one thing right**, do this:

Make the grammar data-driven and inspectable at every stage.

If you want next, I can:

- Design the **node graph UI**
- Specify **expression serialization**
- Define **scope resolution formally**
- Show **how to diff grammars safely**
- Or map this to **DOTS / jobs later**

Just tell me which piece to drill into.



can you give me an example of using this same generator to generate things other than a building (trees or somethign else)

Yes — and this is the **acid test** of whether the system is truly generic.

Below I'll give you **two non-building examples** using the *exact same grammar formalism* and generation pipeline you already defined:

1. A Tree (organic, recursive)
2. A Fence / Wall (linear, modular)

I'll also explicitly call out **what stays the same** and **what changes**, so you can see how one generator kernel supports all of them.

0. What Does *Not* Change

Across **buildings, trees, fences, towns**:

- Same grammar meta-syntax
- Same symbol / rule / expansion system
- Same derivation tree
- Same scopes & overrides
- Same editor UX (select → override → regenerate)
- Same geometry binding phase

Only these change:

- Symbol alphabet
- Rules
- Geometry bindings

That's the goal you're hitting.

PART I — TREE GRAMMAR (Recursive, Organic)

This uses the **same grammar language**, but leans into recursion and probabilistic branching.

1. Tree Grammar (Formal)

1.1 Grammar Header

nginx

```
grammar Tree {  
  
    entry = Tree  
  
    parameters {  
        MaxDepth          = 5  
        BranchLength      = 2.0  
        LengthDecay       = 0.7  
        RadiusDecay       = 0.6  
        BranchChance      = 0.8  
    }  
}
```

1.2 Symbols

less

```
symbols {  
  
    Tree(position: Point)  
  
    Trunk(  
        length: Float,  
        radius: Float,  
        depth: Int  
    )  
  
    Branch(  
        length: Float,  
        radius: Float,  
        depth: Int,  
        angle: Float  
    )  
}
```

```

    LeafCluster(size: Float)
  }

```

1.3 Rules

Root Rule

less

```

rules {

  rule TreeToTrunk
  when Tree(position)
  => Trunk(
    length = BranchLength * 3,
    radius = 0.3,
    depth  = 0
  )
}

```

Trunk Growth

less

```

rule TrunkGrow
when Trunk(length, radius, depth)
if depth < MaxDepth
=> Branch(
  length = length * LengthDecay,
  radius = radius * RadiusDecay,
  depth  = depth + 1,
  angle  = random(-15, 15)
)

```

Branch Splitting (Recursive)

matlab

```

rule BranchSplit
  when Branch(length, radius, depth, angle)
  if depth < MaxDepth
  => choose {

    0.7:
      split Y {
        Branch(
          length = length * LengthDecay,
          radius = radius * RadiusDecay,
          depth  = depth + 1,
          angle  = angle + random(15, 30)
        )
        Branch(
          length = length * LengthDecay,
          radius = radius * RadiusDecay,
          depth  = depth + 1,
          angle  = angle - random(15, 30)
        )
      }

    0.3:
      LeafCluster(size = length)
  }

```

Terminal Leaves

matlab

```

rule LeafEnd
  when Branch(length, radius, depth, angle)
  if depth >= MaxDepth
  => LeafCluster(size = length)
}

```

1.4 Bindings

```
rust
```

```
bindings {  
  Trunk      -> Mesh("TrunkMesh")  
  Branch     -> Mesh("BranchMesh")  
  LeafCluster -> Mesh("LeavesMesh")  
}
```

 Copy code

2. What the Tree Generator Expects as Input

From the editor:

```
yaml
```

Generate:

Symbol: Tree

Position: Vector3

Everything else comes from the grammar.

 Copy code

3. Why This Works in Your System

- Recursion uses **symbol depth**
- Probabilistic growth uses **choose**
- Branch angles are **parameters**, not transforms
- Selection works:
 - Click a branch → override rule → force leaves
- Baking works:
 - Branch meshes instanced
 - Leaves batched

No special-case tree logic required.

PART II — FENCE / WALL GRAMMAR (Linear, Modular)

This is closer to buildings, but *not* volumetric.

1. Fence Grammar

1.1 Grammar Header

nginx

```
grammar Fence {  
  
    entry = Fence  
  
    parameters {  
        PostSpacing = 2.0  
        FenceHeight = 1.2  
    }  
}
```

1.2 Symbols

scss

```
symbols {  
  
    Fence(path: Path)  
  
    Segment(length: Float)  
  
    Post(height: Float)  
    Panel(width: Float, height: Float)  
}
```

1.3 Rules

Path → Segments

lua

```
rules {
```

```
rule FenceToSegments
  when Fence(path)
  => repeat X min PostSpacing {
    Segment(length = PostSpacing)
  }
```

Segment → Post + Panel

arduino

```
rule SegmentToElements
  when Segment(length)
  => split X {
    Post(height = FenceHeight)
    Panel(width = length, height = FenceHeight)
  }
}
```

1.4 Bindings

mathematica

```
bindings {
  Post  -> Mesh("FencePost")
  Panel -> Mesh("FencePanel")
}
```

2. Editor Interaction Example

- Draw a path
- Generator produces fence
- Select some panels → replace with gate rule
- Partial regeneration only affects selected scope

 Copy code

Same system. No new engine code.

PART III — Why This Proves the Design

Let’s compare:

Aspect	Building	Tree	Fence
Entry symbol	Building	Tree	Fence
Path-based	Yes	No	Yes
Recursion	Low	High	None
Split/Repeat	Yes	Yes	Yes
Overrides	Windows → Doors	Branch → Leaves	Panel → Gate
Same engine	✓	✓	✓

This means:

**Your generator is not a building generator.
It is a grammar-driven spatial compiler.**

PART IV — Mental Model (Very Important)

Think of grammars as **programs**:

- Buildings = imperative layout programs
- Trees = recursive growth programs
- Fences = linear tiling programs
- Towns = orchestration programs

All compiled by the same engine.

Where to Go Next (Recommended)

The most valuable next deep dives would be:

1. **Formal override storage & conflict resolution**

2. Node-graph UX for writing these grammars
3. Performance & caching for recursive grammars
4. Town grammar that instantiates buildings
5. How to validate grammars statically

If you want, I can also:

- Stress-test this with caves, ruins, pipes, or interiors
- Show how to constrain trees to grow *away* from buildings
- Design a minimal "standard library" of grammar rules

Just tell me what you want to explore next.



when you say things like "choice" is it user choice or random choice? if possible I'd like the final product to let the user randomize everything or use a default option, and then they can manually go and change features in the editor

This is a **crucial clarification**, and you're asking exactly the right question.

Short answer: **it's both — but they must be *cleanly separated*.**

Below I'll explain **precisely what "choice" means**, how to support **random, default, and user-forced outcomes simultaneously**, and how to implement this *without turning the system into spaghetti*.

1. What "choice" Actually Means (Formally)

In the grammar as written, `choose {}` is a ***rule-selection mechanism*** — not inherently "random" or "user-driven".

Think of `choose` as:

"Multiple valid expansions exist here."

How one is selected is a policy decision, not a grammar decision.

2. Three Distinct Selection Modes (You Need All Three)

Your final system should support **three orthogonal selection modes** at every `choose` site:

2.1 Default (Deterministic)

- Always picks the same option
- Used for:
 - Authoring
 - Debugging
 - Consistent art direction

Example:

css

```
choose {  
    default: Window(...)  
    option: Door(...)  
}
```

2.2 Randomized (Weighted)

- Uses seeded randomness
- Fully reproducible
- Used for variation

 Copy code

Example:

css

```
choose {  
    0.7: Window(...)  
    0.3: Door(...)  
}
```

Important:

- Randomness must be **seeded per scope**
- Rebuilding produces identical results unless seed changes

 Copy code

2.3 User-Forced (Override)

- User explicitly selects outcome in editor
- Overrides randomness and defaults
- Persisted per scope

Example:

```
cpp
```

```
override BayScope_1234 → Door
```

 Copy code

3. How These Modes Interact (Priority Order)

This is **non-negotiable** for sanity.

Selection priority:

1. User override
2. Explicit default
3. Random weighted choice

If you do this in any other order, your editor will feel broken.

4. How This Looks in the Grammar (Formal Extension)

We slightly extend `choose` syntax:

```
arduino
```

```
choose {
```

```
  default:
```

```
    Window(width = width * 0.7, height = FloorHeight * 0.5)
```

```
option weight 0.3:  
  Door(width = width * 0.9, height = FloorHeight * 0.9)  
}
```

Or minimal form (default inferred):

CSS

```
choose {  
  0.7: Window(...)  
  0.3: Door(...)  
}
```

Internally:

- First option becomes default
- Others are alternatives

 Copy code

5. Editor UX: What the User Actually Sees

This is where your design really shines.

5.1 Initial Generation

User clicks:

Generate Building

System:

- Uses default rules
- Or uses seeded random
- No overrides yet

5.2 Randomize Button

User clicks:

Randomize

System:

- Rerolls random choices
- Keeps structure intact
- Does *not* erase overrides

5.3 Manual Editing

User:

- Clicks windows
- Sees:

sql

Feature: Bay

Current Rule: Window

Alternatives:

- Window
 - Door
- Selects Door

System:

- Stores override at that scope
- Locks that choice

 Copy code

5.4 Reset to Grammar

User clicks:

Reset Selection to Grammar

System:

- Removes override
- Falls back to default or random

6. How to Store This Cleanly (Serialization)

You **do not** store the chosen result directly.

You store:

```
java
```

```
ChoiceResolution {
    ScopeID
    RuleID
    SelectedOptionID
    SelectionMode = UserForced
}
```

Random selections:

- Not stored
- Recomputed from seed

Defaults:

- Encoded in grammar

 Copy code

7. Seed Strategy (Critical Detail)

Randomization must be:

- Deterministic
- Scope-local
- Stable across regenerations

Good seed example:

```
ini
```

```
Seed = hash(GlobalSeed, ScopeID, RuleID)
```

This guarantees:

- Changing one floor doesn't reroll the whole building
- Partial regeneration works

 Copy code

8. Example: Bay Rule Revisited (Fully Correct)

arduino

```
rule GroundBay
  when Bay(width, floorIndex)
  if floorIndex == 0
  => choose {

    default:
      Window(width = width * 0.8, height = FloorHeight * 0.6)

    option weight 0.4:
      Door(width = width * 0.9, height = FloorHeight * 0.9)
  }
```

User flow:

1. Generate → all windows
2. Randomize → some doors
3. Select → force door
4. Regenerate → door stays

9. Why This Design Is Robust

- ✓ Designers get sane defaults
- ✓ Artists get controlled randomness
- ✓ Users get manual control
- ✓ Overrides never get lost
- ✓ Grammar remains pure
- ✓ Editor remains predictable

 Copy code

10. Important Anti-Patterns to Avoid

- ✗ "Random inside grammar code"
- ✗ "Storing results instead of decisions"
- ✗ "User edits directly modify meshes"
- ✗ "Overrides without scope IDs"

Final Mental Model (Lock This In)

Grammar defines *possibility space*.

Randomization explores it.

User overrides collapse it locally.

If you want, next I can:

- Design the **override resolution algorithm**
- Show how **undo/redo** integrates
- Specify the **editor UI panels**
- Explain **how to preview alternatives**
- Formalize **conflict resolution** when multiple rules apply

Just say where you want to go next.



how would we ensure that things dont break due to the grammar, whether it be conflicting rules, invalid entries, etc. rather, how would the code pipeline work. i am aware that compilers do lexing, parsing, ast, etc as its process, is there a structure we could follow to streamline computation very very nicely

This is *exact*ly the right way to think about this.

What you're really asking is:

"How do I turn this grammar system into a *compiler pipeline* so errors are impossible (or at least well-contained), generation is fast, and editor feedback is excellent?"

Yes — there is a **very clean, compiler-inspired structure** that fits your system *perfectly*.

Below I'll give you:

1. A **full compiler-style pipeline** tailored to procedural grammars
2. What invariants are enforced at each stage
3. How this prevents breakage, conflicts, and undefined behavior
4. How to integrate this into Unity editor workflows cleanly

This is long, but this is the *right* mental model.

1. High-Level Pipeline Overview

Your procedural generator should operate as a **multi-stage compiler**, not a “script runner”.

Conceptual Pipeline

mathematica

Grammar Source



Lexing / Loading



Parsing



Semantic Analysis



Grammar IR (Validated)



Derivation Planning



Spatial Interpretation



Geometry Binding



Bake / Export

Each stage has:

- A very *narrow responsibility*
- Well-defined inputs/outputs
- Strict invariants

This is what keeps the system robust.

2. Stage-by-Stage Breakdown (With Guarantees)

Stage 0 — Source Acquisition (Editor Layer)

Inputs

- ScriptableObjects
- Optional DSL text
- Editor overrides

 Copy code

Outputs

- Raw grammar data structures

Invariants

- No code execution
- No geometry creation
- No randomness

This is *pure data ingestion*.

Stage 1 — Lexing / Loading

Purpose

Normalize input into a uniform token stream or object graph.

If DSL

- Tokenize keywords, symbols, literals

If ScriptableObjects

- Convert fields → tokens / nodes

Output

- Token stream or raw parse nodes

Errors caught

- Missing fields
- Invalid identifiers
- Duplicate symbol names

Key rule

Grammar must be *structurally complete* before moving on.

Stage 2 — Parsing (Syntax Validation)

Purpose

Ensure grammar obeys formal structure.

Produces

- Parse tree (or CST)

Validates

- Grammar structure
- Rule syntax
- Expression syntax
- Operator validity

Errors caught

- Malformed rules
- Invalid expansions
- Broken choose blocks
- Bad expressions

Hard rule

No symbol resolution happens yet.

Stage 3 — Semantic Analysis (Most Important)

This is where *90% of bugs are eliminated*.

Purpose

Validate **meaning**, not syntax.

Produces

- Semantic grammar graph

- Symbol table
- Rule table
- Expression trees (typed)

3.1 Symbol Resolution

Validates:

- All symbols referenced exist
- Parameter names match definitions
- Types are compatible

Errors caught

- `Window(width, height)` but symbol only defines `width`
- Using `floorIndex` where none exists

3.2 Rule Conflict Detection

This is where compiler theory shines.

For each symbol:

- Collect all applicable rules
- Analyze predicates
- Check for:
 - Overlapping conditions
 - Unreachable rules
 - Ambiguous rules without priority

Example error:

Two rules match `Facade(width, height, floorIndex)` with no disambiguation

3.3 Choice Validation

Ensure:

- `choose` has ≥ 1 option

- Weights are valid
- Default exists or is inferred
- All branches produce compatible symbols

3.4 Recursion & Termination Checks

Critical for trees.

- Detect unbounded recursion
- Enforce:
 - Depth limits
 - Probability decay
 - Explicit termination rules

Example error:

Branch → Branch without depth change

3.5 Attribute Flow Analysis

Like compiler dataflow analysis.

Ensure:

- All required parameters are initialized
- No circular dependencies
- Parent attributes exist when accessed

Output of Stage 3

A Grammar IR:

- Fully resolved
- Typed
- Conflict-free
- Ready to execute

No editor regeneration should proceed without this.

3. Grammar IR (Intermediate Representation)

This is your internal "bytecode".

Properties

- No strings
- No user text
- All symbols resolved to IDs
- Expressions pre-compiled
- Rules ordered by priority

Why this matters:

- Fast execution
- Deterministic behavior
- Easy caching

4. Stage 4 — Derivation Planning (Execution Phase 1)

Purpose

Evaluate grammar into a **derivation tree**.

Inputs

- Grammar IR
- Entry symbol
- Parameters
- Random seed

Output

- Derivation tree (symbol instances)

Guarantees

- Deterministic
- Scope IDs assigned
- Overrides applied
- No geometry yet

Conflict Safety

- Rule resolution already proven safe
- Overrides applied by exact scope ID
- Randomness is scoped & seeded

5. Stage 5 — Spatial Interpretation (Execution Phase 2)

Purpose

Turn derivation tree into **layout graph**.

Validations

- Splits fit within bounds
- Repeats terminate
- No negative sizes
- No invalid transforms

This stage can:

- Clamp values
- Emit warnings (not errors)

6. Stage 6 — Geometry Binding

Purpose

Attach meshes/materials.

Validations

- Mesh exists
- LODs available
- Material compatibility

Failures here:

- Are *recoverable*
- Use fallback meshes

7. Stage 7 — Baking / Export

Final collapse:

- Mesh merge
- Instancing
- UVs
- FBX export

At this point:

- Grammar is irrelevant
- Output is static

8. Error Handling Philosophy (Very Important)

Errors are categorized:

Stage	Error Type	Result
Parsing	Syntax	Block generation
Semantic	Logic	Block generation
Derivation	Runtime	Block subtree
Layout	Geometry	Clamp + warn
Binding	Asset	Fallback
Baking	Export	Fail gracefully

This keeps editor stable.

9. Incremental Recompilation (Editor Performance)

Key idea:

Grammar changes $\neq$ regenerate everything.

Use:

- Hash grammar IR
- Cache derivation subtrees
- Invalidate only affected scopes

This mirrors **incremental compilation**.

10. Why This Compiler Model Is Perfect for Unity

- Unity already uses ScriptableObject serialization
- Editor feedback is immediate
- Errors point to grammar assets, not runtime
- No mysterious runtime bugs

11. Mental Model (Lock This In)

You are building a compiler that emits geometry instead of machine code.

Everything:

- Grammars = programs
- Rules = functions
- Symbols = types
- Derivation = execution
- Meshes = output artifacts

Once you think this way, *everything* falls into place.

If You Want to Go Even Deeper

Next logical dives:

- Formal conflict-detection algorithms

- Static recursion analysis
- Grammar linting rules
- Visual error reporting in Unity
- DOTS/job-friendly execution model

Just tell me where you want to zoom in next.



please summarize everything we've talked about so far in terms of the theory/grammar end of it (getting it up until we are ready to use unity)

Below is a **clean, theory-first summary** of everything we've discussed **up to (but not including) Unity-specific implementation**.

Think of this as the **design document for the "procedural grammar compiler" itself**.

I'll structure this the way a compiler or PL paper would: **goals → model → grammar → execution → correctness → guarantees**.

1. What You Are Building (Abstractly)

You are building a **general-purpose, grammar-driven spatial compiler**.

Formally:

A system that takes a **parameterized attribute grammar**, evaluates it into a **derivation tree**, interprets that tree as **spatial layout operations**, and later (outside this scope) binds geometry.

Key non-goals (at this stage):

- No meshes
- No rendering
- No Unity concepts
- No editor UX

Just rules → structure → layout.

2. Core Design Goals (Why This Exists)

The system must:

1. Be **domain-agnostic**
 - Buildings, trees, fences, towns, caves, etc.
2. Support **formal grammars**
 - Deterministic, analyzable, debuggable
3. Allow **controlled randomness**
 - Fully reproducible
4. Allow **user overrides**
 - Manual collapse of ambiguity
5. Support **partial regeneration**
 - Changes don't ripple unnecessarily
6. Be **statically verifiable**
 - Most errors caught *before* execution
7. Be **composable**
 - Generators can invoke generators

These goals dictate every theoretical choice.

3. Grammar Model (What Kind of Grammar This Is)

This is **not** a CFG or L-system.

Formally, you are using a:

Parameterized, attribute-driven shape grammar with scoped rule overrides

Key properties:

- **Symbols**
 - Typed
 - Parameterized
- **Rules**

- Conditional
- Possibly probabilistic
- **Attributes**
 - Flow from parents → children
- **Spatial operators**
 - Split, repeat, extrude, etc.
- **Scopes**
 - Semantic groupings for selection and overrides

This is closest to **CGA shape grammars**, but extended for:

- Interactivity
- Incrementality
- Editor tooling

4. Fundamental Concepts (Vocabulary)

These are the *atoms* of the system.

4.1 Symbols

A **symbol** is a semantic instruction, not geometry.

Formally:

CSS

Symbol := (Type, Parameters)

Examples:

- Facade(width, height, floorIndex)
- Branch(length, radius, depth)

 Copy code

Symbols:

- Are nodes in a derivation tree
- Carry parameters
- Belong to exactly one scope

4.2 Rules

A **rule** defines how a symbol expands.

Form:

mathematica

Rule := (InputSymbol, Condition?, Expansion)

Rules:

- Are mutually exclusive or explicitly prioritized
- May produce multiple symbols
- Never directly create geometry

 Copy code

4.3 Expansions (Production Operators)

Rules expand symbols using **structured operators**, not arbitrary code:

- **Production**: symbol → symbol
- **Split**: divide space into parts
- **Repeat**: tile symbols along an axis
- **Choose**: declare alternatives

These operators are *semantic*, not procedural.

4.4 Choose ≠ Random

choose {} means:

"Multiple valid expansions exist."

Selection policy is separate.

Selection modes:

1. **Default** (deterministic)
2. **Random** (seeded, weighted)

3. User override (forced)

Priority order:

```
rust
```

User **override** > **Default** > Random

This separation is critical for correctness.

 Copy code

4.5 Scopes

Scopes are **semantic groupings** of symbols.

Examples:

- Building
- Floor
- Facade
- Row of windows

Scopes enable:

- Feature selection
- Local overrides
- Partial regeneration

Every symbol has:

```
css
```

Symbol → Scope → ParentScope → ...

 Copy code

4.6 Derivation Tree

Grammar execution produces a **derivation tree**, not geometry.

Properties:

- Nodes = symbol instances

- Stable IDs
- Parent/child relationships
- Rule provenance recorded

This tree is:

- Inspectable
- Editable via overrides
- Incrementally regenerable

5. Execution Model (How Grammars Run)

Grammar execution is **compiler-like**, not script-like.

5.1 Two-Phase Execution

Phase 1 — Derivation

- Apply grammar rules
- Produce derivation tree
- No spatial reasoning yet

Phase 2 — Spatial Interpretation

- Interpret symbols as layout operations
- Produce abstract geometry (boxes, faces, segments)
- Still no meshes

This separation is mandatory for robustness.

5.2 Determinism & Randomness

Randomness is:

- Seeded
- Scoped
- Deterministic

Seed example:

ini

```
Seed = hash(GlobalSeed, ScopeID, RuleID)
```

Guarantees:

- Regeneration is stable
- Partial regeneration doesn't reshuffle everything
- Overrides remain intact

 Copy code

6. Overrides (User Interaction Model, Abstractly)

Overrides are **not** edits to geometry.

They are:

```
go
```

```
Override := (ScopeID, RuleID, SelectedOption)
```

Properties:

- Persist independently of regeneration
- Override only local behavior
- Remove ambiguity at that scope

 Copy code

This is what allows:

- Manual edits
- Designer control
- Safe iteration

7. Composability (Generators Within Generators)

A generator is just a grammar whose **entry symbol** is used by another grammar.

Formally:

- Grammars emit symbols
- Some symbols invoke sub-grammars

- Communication is via attributes only

Rules:

- No parent inspects child geometry
- No cross-grammar mutation
- Deterministic boundaries

This enables:

- Buildings → towns
- Branches → trees
- Rooms → interiors

8. Correctness & Safety (Why This Doesn't Break)

The system enforces correctness via a **compiler pipeline**.

8.1 Grammar Compilation Stages

1. Loading

- Raw data ingestion

2. Parsing

- Structural correctness

3. Semantic Analysis

- Symbol resolution
- Rule conflict detection
- Attribute flow validation
- Recursion termination analysis

4. Grammar IR

- Fully validated internal form

Only after this does execution occur.

8.2 Static Guarantees

Before execution:

- No missing symbols
- No ambiguous rules
- No invalid parameters
- No unbounded recursion
- No invalid choose blocks

This eliminates most runtime failures.

9. Grammar IR (Intermediate Representation)

The grammar is compiled into an IR:

Properties:

- No strings
- All IDs resolved
- Typed expressions
- Rules pre-ordered
- Ready for fast execution

This is equivalent to bytecode in a compiler.

10. What Exists *Before* Unity

At the point you stop (right before Unity):

You have:

- A formal grammar language
- A compiler pipeline
- A validated grammar IR
- A derivation executor
- A spatial layout interpreter
- Deterministic randomness
- Override semantics
- Composability guarantees

You **do not yet care about**:

- Meshes
- Rendering
- Inspector UI
- Serialization formats
- Performance tuning

Those come next.

11. The Mental Model (This Is the Core Insight)

You are not generating buildings.

You are compiling spatial programs into structure.

Everything else — Unity, meshes, FBX, editor tools — is just a *backend*.

If you want, the *natural* next steps after this summary are:

1. Mapping this theory cleanly onto **Unity ScriptableObject** data
2. Designing the **visual grammar editor**
3. Formalizing **scope selection & override resolution**
4. Designing the **geometry binding backend**

Just tell me which direction you want to go next.



continuing the compiler analogy, would this be like im providing a path as the "high level code" and the compiler takes that and turns it into the mesh as the "bytecode"

Yes — that analogy is not only correct, it's the *right* way to think about the entire system.

With one important refinement.

Short version:

- The path is *not* the program.
- The path is *input data* to the program.
- The grammar is the program.

Let’s lock this mental model in precisely, because it will guide every correct architectural decision you make.

1. The Correct Compiler Analogy (Fully Mapped)

Here is the clean mapping:

Compiler Concept	Your System
Source code	Grammar definition
Program arguments	User inputs (paths, seeds, parameters)
Compiler	Grammar engine
AST	Derivation tree
IR	Spatial layout graph
Backend	Geometry / mesh builder
Bytecode / machine code	Final mesh (FBX)

So the answer is **almost yes**, but with a critical distinction.

2. Where the Path Actually Fits

The path (or spline, polyline, footprint, etc.) is:

- Runtime input to a compiled grammar program

Exactly like:

```
bash
```

```
myCompiler program.c --input=data.txt
```

So:

 Copy code

- The **grammar** defines *what kind of thing* can be built
- The **path** constrains *where and how it manifests*

Example Mapping

Grammar (program):

```
nginx
```

Building → Floors → Facades → Bays → Features

Input (arguments):

 Copy code

- Footprint path
- Height
- Style seed
- User overrides

Output:

- A concrete building mesh

The grammar is reusable across many paths — just like compiled code.

3. The Full Pipeline (Compiler-Accurate)

Let's rewrite your system using *strict compiler language*:

```
css
```

```
Grammar Source
```

```
↓
```

```
[ Grammar Compiler ]
```

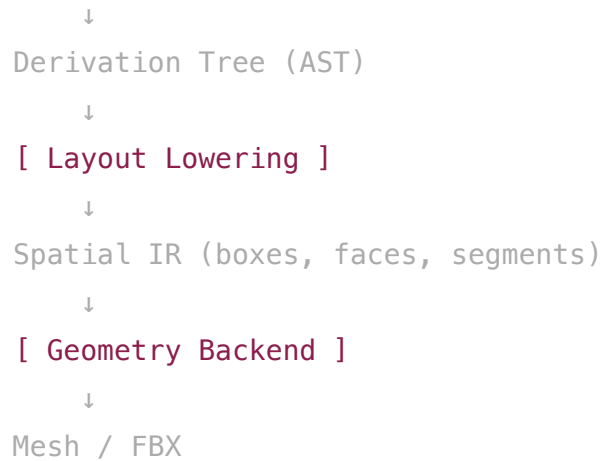
```
↓
```

```
Validated Grammar IR
```

```
↓
```

```
[ Grammar Interpreter ]
```

```
+ Runtime Inputs (path, params, seed)
```



This is **exactly** how a modern compiler works.

4. Why the Path Is *Not* High-Level Code

If the path were “code”, you’d have problems:

- Paths don’t express intent
- Paths don’t encode rules
- Paths shouldn’t need validation like grammars
- You’d lose reusability

Instead:

The grammar is the *algorithm*.

The path is *data*.

Just like:

```
c
```

```
sort(array)
```

Where:

- `sort` = grammar
- `array` = path

5. Multiple Inputs, Same Grammar

This model scales perfectly:

Same grammar:

- Straight building
- Curved building
- Courtyard building
- Multiple footprints

Different inputs:

- Path
- Height
- Zoning rules
- User overrides

All go through the same compiled grammar.

6. Editor Actions = Recompilation vs Rerun

This also cleanly separates editor actions:

Change grammar

- **Recompile grammar**
- Invalidates all derived structures

Change path

- **Rerun grammar**
- Same grammar IR, new derivation tree

Change override

- **Patch AST**
- Partial regeneration only

This is *exactly* how IDEs work.

7. What “Bytecode” Actually Is Here

The mesh is not quite bytecode — it's more like **final machine code**.

A closer analogy:

Compiler

Your System

AST

Derivation tree

SSA / IR

Spatial layout graph

Instruction selection

Geometry binding

Machine code

Vertex/index buffers

FBX is just a **serialization format** for the result.

8. Why This Mental Model Is So Powerful

Once you adopt this:

- Validation becomes obvious
- Debugging becomes inspectable
- Partial rebuilds are natural
- Editor tooling becomes IDE tooling
- Performance optimizations have clear layers

You stop asking:

"How do I generate buildings?"

And start asking:

"How do I compile spatial programs efficiently?"

That's the right question.

9. One Final Refinement (Important)

You actually have **two "languages"**:

1. **Grammar language** (author-facing)

2. Spatial IR (engine-facing)

Unity never sees the grammar language directly.

It only sees the *lowered result*.

That separation is what keeps this robust.

10. The One-Sentence Takeaway

You are writing a compiler where grammars are programs, paths are runtime inputs, and meshes are machine code.

If you want, next we can:

- Define what your **Spatial IR** should look like
- Design **debug tooling** (breakpoints in grammars!)
- Discuss **incremental recompilation**
- Map this cleanly onto Unity's asset system

Just tell me where to go next.

